Exercise Sheet 11

Efficient Computing/Parallel Computing 2 in SS 2026

Date of issue: 2.7.2026 Deadline: 10.7.2026

The exercises marked with ★ are relevant for admission to the examination and must be submitted in electronic form in Moodle by the submission deadline.

Exercise 11.1 (*Roofline Analysis of a Direct N-Body Simulation*).

We consider the classical direct N-body simulation in 2D. In each time step, the force contribution of all bodies j is summed for each body i. A *naive* CUDA kernel (one thread per body i) is given below. In each j-iteration, the coordinates and the mass of body j are loaded from global memory, i.e., there is no data reuse across threads.

```
__global__ void nbody_naive(const float* x, const float* y,
    const float* m, float* vx, float* vy, int N,
    float G, float dt){
  int i = blockIdx.x * blockDim.x + threadIdx.x;
  if (i >= N) return;
  float xi = x[i], yi = y[i]; /* held in registers */
  float ax = 0.f, ay = 0.f;
  for (int j = 0; j < N; ++j) {
    float dx = x[j] - xi;     /* GLOBAL load x[j] */
    float dy = y[j] - yi;     /* GLOBAL load y[j] */
    float dist = dx*dx + dy*dy;
    if (dist > 1e-8f && i != j) {
      float invDist  = rsqrtf(dist);
      float invDist3 = invDist*invDist*invDist;
      float f = G * m[j] * invDist3; /* GLOBAL load m[j] */
      ax += dt * dx * f;
      ay += dt * dy * f;
    }
  }
  vx[i] += ax; vy[i] += ay;
}
```

A *tiled* variant (tiling) caches a block of T bodies in shared memory and reuses each cached body for all T threads of the block:

```
#define T 128
__global__ void nbody_tiled(const float* x, const float* y,
    const float* m, float* vx, float* vy, int N,
    float G, float dt){
  __shared__ float sx[T], sy[T], sm[T];
  int i = blockIdx.x * blockDim.x + threadIdx.x;
  float xi = (i<N)? x[i]:0.f, yi = (i<N)? y[i]:0.f;
  float ax = 0.f, ay = 0.f;
  for (int tile = 0; tile < gridDim.x; ++tile) {
```



```
    int j = tile*T + threadIdx.x;
  /* one GLOBAL load per thread */
    sx[threadIdx.x] = (j<N)? x[j]:0.f;
    sy[threadIdx.x] = (j<N)? y[j]:0.f;
    sm[threadIdx.x] = (j<N)? m[j]:0.f;
    __syncthreads();
    #pragma unroll
    for (int k = 0; k < T; ++k) {
     /* operands from SHARED memory */
      float dx = sx[k] - xi, dy = sy[k] - yi;
      float dist = dx*dx + dy*dy;
      if (dist > 1e-8f) {
        float invDist  = rsqrtf(dist);
        float invDist3 = invDist*invDist*invDist;
        float f = G * sm[k] * invDist3;
        ax += dt*dx*f; ay += dt*dy*f;
      }
    }
    __syncthreads();
  }
  if (i<N){ vx[i]+=ax; vy[i]+=ay; }
}
```

Perform a roofline analysis for an **NVIDIA A100** with FP32 peak $P_{\text{peak}} = 19.5$ TFLOP/s and HBM2 bandwidth $B_{\text{peak}} = 1.5$ TB/s. Count each basic operation $(+, -, *)$ as 1 FLOP and `rsqrtf` as 1 FLOP.

a) Count the FLOPs per (i, j) interaction.

b) For the *naive* kernel: estimate the global memory traffic per interaction, compute the arithmetic intensity $I = \text{FLOP/Byte}$, the achievable performance $P = \min(P_{\text{peak}}, B_{\text{peak}} I)$, and classify the kernel as memory-bound or compute-bound.

c) For the *tiled* kernel ($T = 128$): explain the reduction in global traffic, recompute I and P, and classify the kernel as memory-bound or compute-bound.

d) Determine the roofline ridge point $I^* = P_{\text{peak}}/B_{\text{peak}}$ and describe the horizontal shift of the operating point due to tiling.

e) Measure the achieved performance (GFLOP/s) of both kernels launched with `blockDim.x=T` and `gridDim.x=`$\lceil N/T \rceil$ for $N = 65536$ using CUDA events. Plot both points on the roofline chart from part d, and explain any gap to the theoretical prediction.

